

Capitolo 3 - Crittografia Simmetrica e Confidenzialità

La crittografia simmetrica, definita anche crittografia convenzionale, a chiave segreta o a chiave singola, era l'unico tipo di crittografia in uso prima dello sviluppo della crittografia a chiave pubblica alla fine degli anni '70. Rimane di gran lunga la più ampiamente utilizzata tra i due tipi di crittografia.

- **Testo in chiaro:** Questo è il messaggio originale o i dati che vengono inseriti nell'algoritmo come input.
- **Algoritmo di crittografia:** L'algoritmo di crittografia esegue varie sostituzioni e trasformazioni sul testo in chiaro.
- **Chiave segreta:** La chiave segreta è anch'essa un input per l'algoritmo. Le esatte sostituzioni e trasformazioni eseguite dall'algoritmo dipendono dalla chiave.
- **Testo cifrato:** Questo è il messaggio rimescolato prodotto come output. Dipende dal testo in chiaro e dalla chiave segreta. Per un dato messaggio, due chiavi diverse produrranno due testi cifrati diversi.
- **Algoritmo di de-crittografia:** Questo è essenzialmente l'algoritmo di crittografia eseguito al contrario. Prende il testo cifrato e la stessa chiave segreta e produce il testo in chiaro originale.

I sistemi crittografici sono genericamente classificati secondo tre dimensioni indipendenti:

1. **Il tipo di operazioni utilizzate per trasformare il testo in chiaro in testo cifrato.** Tutti gli algoritmi di crittografia si basano su due principi generali: la **sostituzione**, in cui ogni elemento del testo in chiaro (bit, lettera, gruppo di bit o lettere) viene mappato in un altro elemento, e la **trasposizione**, in cui gli elementi del testo in chiaro vengono riorganizzati. Il requisito fondamentale è che nessuna informazione vada persa (ovvero che tutte le operazioni siano reversibili).
2. **Il numero di chiavi utilizzate.** Se sia il mittente che il ricevente utilizzano la stessa chiave, il sistema è definito simmetrico, a chiave singola, a chiave segreta o crittografia convenzionale. Se il mittente e il ricevente utilizzano ciascuno una chiave diversa, il sistema è definito asimmetrico, a due chiavi o crittografia a chiave pubblica.
3. **Il modo in cui viene elaborato il testo in chiaro.** Un **cifrario a blocchi** elabora l'input un blocco di elementi alla volta, producendo un blocco di output per ogni blocco di input. Un **cifrario a flusso** elabora gli elementi di input continuamente, producendo un elemento di output alla volta, man mano che procede.

Struttura della cifratura a blocchi - Cifrario di Feistel

Il cifrario a blocchi simmetrico consiste in una sequenza di cicli con sostituzioni e permutazioni controllate dalla chiave.

Questa regola è stata descritta per la prima volta da Horst Feistel nel 1973 quando propose il suo omonimo cifrario.

Gli input per l'algoritmo di crittografia sono un blocco di testo in chiaro di lunghezza $2w$ bit e una chiave K . Il blocco di testo in chiaro è diviso in due metà:

$$L_0 \text{ e } R_0$$

Le due metà dei dati passano attraverso n round di elaborazione e poi si combinano per produrre il blocco di testo cifrato. Ogni round i ha come input:

$$L_{i-1} \text{ e } R_{i-1}$$

derivati dal round precedente, così come una sottochiave:

$$K_i$$

derivata dalla chiave complessiva K . In generale, le sotto-chiavi sono diverse da K e l'una dall'altra, e sono generate dalla chiave mediante un algoritmo di generazione delle sotto-chiavi.

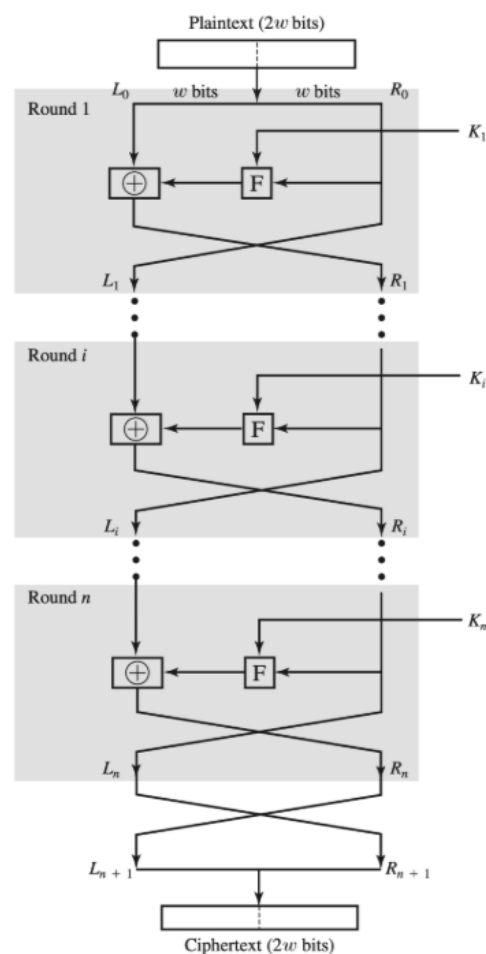


Figure 20.1 Classical Feistel Network

Tutti i round hanno la stessa struttura. Viene eseguita una sostituzione sulla metà sinistra dei dati. Ciò avviene applicando una **funzione di round F** alla metà destra dei dati e prendendo poi l'OR esclusivo (XOR) dell'output di quella funzione e della metà sinistra dei dati. La funzione di round ha la stessa struttura generale per ogni round ma è parametrizzata dalla sottochiave di round K_i . In seguito a questa sostituzione, viene eseguita una permutazione che consiste nell'interscambio delle due metà dei dati.

La struttura di Feistel è un esempio particolare della struttura più generale utilizzata da tutti i cifrari simmetrici a blocchi. In generale, un cifrario simmetrico a blocchi consiste in una sequenza di round, con ogni round che esegue sostituzioni e permutazioni condizionate da un valore di chiave segreta. L'esatta realizzazione di un cifrario simmetrico a blocchi dipende dalla scelta dei seguenti parametri e caratteristiche di progettazione:

- **Dimensione del blocco:** Dimensioni del blocco più grandi significano maggiore sicurezza (a parità di altre condizioni) ma ridotta velocità di crittografia/decrittografia. Una dimensione del blocco di 128 bit è un compromesso ragionevole ed è quasi universale tra i recenti progetti di cifrari a blocchi.
- **Dimensione della chiave:** Una dimensione della chiave più grande significa maggiore sicurezza ma può diminuire la velocità di crittografia/decrittografia. La lunghezza della chiave più comune negli algoritmi moderni è 128 bit.
- **Numero di round:** L'essenza di un cifrario simmetrico a blocchi è che un singolo round offre una sicurezza inadeguata, ma che round multipli offrono una sicurezza crescente. Una dimensione tipica è 16 round.
- **Algoritmo di generazione delle sottochiavi:** Una maggiore complessità in questo algoritmo dovrebbe portare a una maggiore difficoltà della crittoanalisi.

- **Funzione di round:** Ancora una volta, una maggiore complessità significa generalmente una maggiore resistenza alla crittoanalisi.

Algoritmi di cifratura simmetrica

Gli algoritmi di crittografia simmetrica più comunemente usati sono i cifrari a blocchi. Un cifrario a blocchi elabora l'input di testo in chiaro in blocchi di dimensione fissa e produce un blocco di testo cifrato di pari dimensione per ogni blocco di testo in chiaro. Questa sezione e la prossima si concentrano sui tre più importanti cifrari a blocchi simmetrici: il Data Encryption Standard (DES), il triple DES (3DES) e l'Advanced Encryption Standard (AES).

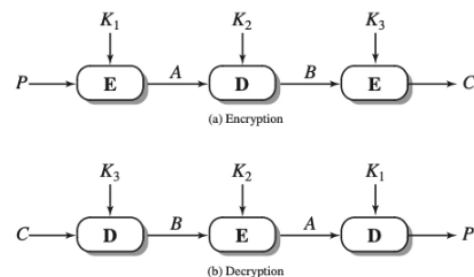
DES

L'algoritmo DES ha un testo in chiaro è lungo 64 bit e la chiave è lunga 56 bit; quantità di testo in chiaro più lunghe vengono elaborate in blocchi da 64 bit. La struttura DES è una variante minore della rete di **Feistel**. Sono previsti 16 round di elaborazione. Nella fase di cifratura, dalla chiave originale a 56 bit vengono generate 16 **sottochiavi**, una delle quali viene utilizzata per ogni round. Il processo di **decrittografia** con DES è essenzialmente lo stesso del processo di crittografia. La regola è la seguente: utilizzare il testo cifrato come input per l'algoritmo DES, ma utilizzare le **sottochiavi** in ordine inverso.

Triple DES

Il 3DES utilizza tre chiavi e tre esecuzioni dell'algoritmo DES. La funzione segue una sequenza **encrypt-decrypt-encrypt** (EDE):

- **Fase 1 (Criptazione):** Il testo in chiaro viene cifrato con la prima chiave (K_1).
- **Fase 2 (Decriptazione):** Il risultato viene "decryptato" usando una seconda chiave (K_2). Poiché K_2 è diversa da K_1 , questa operazione non riporta al testo originale, ma crea un ulteriore livello di confusione (**scrambling**).
- **Fase 3 (Criptazione):** Il risultato viene cifrato un'ultima volta con la terza chiave (K_3).



$$C = E(K_3, D(K_2, E(K_1, P)))$$

$$P = D(K_1, E(K_2, D(K_3, C)))$$

La **decrittografia** è semplicemente la stessa operazione con le chiavi invertite.

Non vi è alcun significato crittografico nell'uso della **decrittografia** per la seconda fase della crittografia 3DES. Il suo unico vantaggio è che consente agli utenti del 3DES di **decrittografare** i dati **crittografati** dagli utenti del vecchio DES singolo.

AES

È destinato a sostituire il DES e il triple DES con un algoritmo più sicuro ed efficiente. A differenza del DES, l'AES **non utilizza una struttura di Feistel**. Mentre i vecchi algoritmi dividevano il blocco a metà elaborandone solo una parte alla volta, l'AES elabora l'intero blocco di dati in parallelo.

- **Dimensione del Blocco:** 128 bit (fissa).
- **Lunghezza della Chiave:** Può essere di 128, 192 o 256 bit.
- **Lo "State":** I dati vengono organizzati in una matrice quadrata 4x4 di byte chiamata **State**. Tutte le operazioni agiscono su questa "scacchiera" di 16 byte.
- **Iterazioni (Round):** Il numero di round dipende dalla chiave:
 - 10 (per 128 bit)
 - 12 (per 192 bit)
 - 14 (per 256 bit).

Rappresentazione dei Dati: Lo "State"

I dati in input (128 bit) vengono organizzati in una **matrice quadrata 4x4 di byte** chiamata **State**. Tutte le trasformazioni dell'algoritmo operano su questa "scacchiera" di 16 byte. Anche la chiave viene visualizzata come una matrice di byte e poi espansa.

Espansione della Chiave

L'algoritmo non usa la chiave originale così com'è in ogni round. La chiave iniziale (es. 128 bit) viene espansa in un array di 44 parole da 32 bit attraverso un algoritmo dedicato. Questo crea una **sotto-chiave unica per ogni round**, aumentando drasticamente la resistenza agli attacchi.

Le Quattro Fasi di un Round

Ogni round (eccetto l'ultimo) applica quattro trasformazioni matematiche per garantire **confusione e diffusione**.

1. Substitute Bytes (Sostituzione)

Questa è una fase di sostituzione non lineare che opera indipendentemente su ogni byte dello *State*.

- **Il Meccanismo:** Si utilizza una tabella di ricerca predefinita chiamata **S-box**, che consiste in una matrice 16x16 contenente una permutazione di tutti i possibili 256 valori a 8 bit.
- **Mappatura:** Per ogni byte, i 4 bit di sinistra (MSB) determinano la **riga** e i 4 bit di destra (LSB) determinano la **colonna** della S-box.
- **Esempio:** Se un byte ha valore esadecimale **{95}**, si incrocia la riga 9 con la colonna 5, ottenendo il valore **{2A}**.
- **Obiettivo:** È l'unica fase non lineare dell'AES. È progettata per avere una bassa correlazione tra bit di input e output, rendendo l'algoritmo resistente alla crittoanalisi statistica.

Table 20.2 AES S-Boxes

(a) S-box

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	82	75
4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
B	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

2. Shift Rows (Scorrimento Righe)

Questa è una fase di permutazione che sposta i byte all'interno delle righe dello *State*.

87	F2	4D	97
EC	6E	4C	90
4A	C3	46	E7
8C	D8	95	A6

→

87	F2	4D	97
6E	4C	90	EC
46	E7	4A	C3
A6	8C	D8	95

- **Procedimento:** Le righe della matrice vengono fatte scorrere circolarmente verso sinistra con offset differenti:
 - **Riga 1:** Non subisce alcuna modifica.
 - **Riga 2:** Scorrimento circolare a sinistra di **1 byte**.
 - **Riga 3:** Scorrimento circolare a sinistra di **2 byte**.
 - **Riga 4:** Scorrimento circolare a sinistra di **3 byte**.
- **Impatto:** Questa trasformazione assicura che i quattro byte di una singola colonna vengano distribuiti in quattro colonne diverse per il round successivo.
- **Diffusione:** Sposta i dati tra le colonne, garantendo quella che i tuoi appunti chiamano "diffusione orizzontale".

3. Mix Columns (Miscelazione colonne)

Questa fase opera sulle colonne dello *State*, trattando ogni colonna come un vettore matematico.

- **Il Calcolo:** Ogni byte di una colonna viene mappato in un nuovo valore che è funzione di **tutti e quattro** i byte della colonna stessa. Matematicamente, questo viene calcolato usando equazioni su **campi finiti**.
- **Proprietà:** È progettata per fornire un'ottima miscelazione dei bit.
- **Diffusione Totale:** Combinata con lo *Shift Rows*, garantisce che dopo pochi round ogni bit di output dipenda potenzialmente da ogni singolo bit di input (effetto valanga).

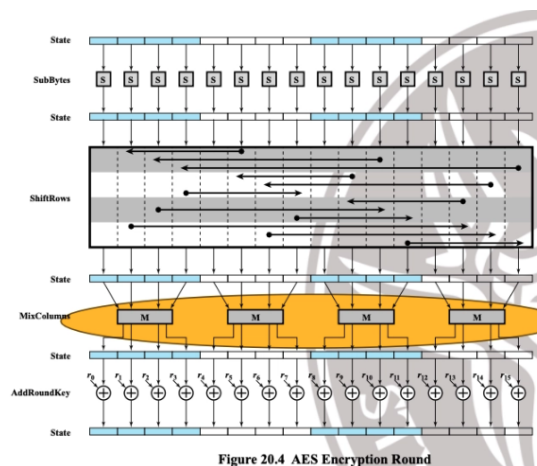


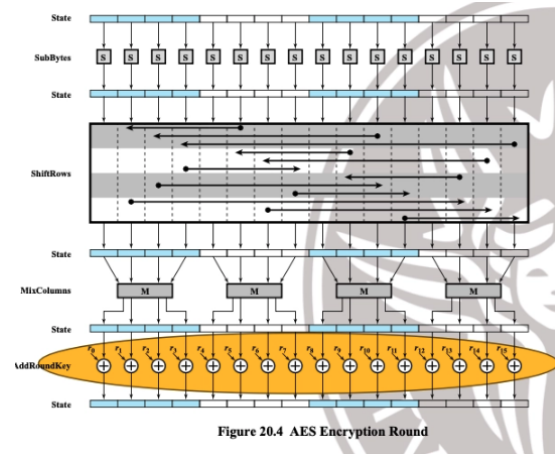
Figure 20.4 AES Encryption Round

- **Eccezione:** Questa fase **non viene eseguita** nell'ultimo round (il decimo round per AES-128) per rendere il cifrario correttamente reversibile durante la decrittazione.

4. Add Round Key (Aggiunta Chiave)

È l'operazione più semplice ma la più cruciale, poiché è l'unica che introduce la segretezza della chiave.

- **Operazione:** Viene eseguito un **XOR bit a bit** tra i 128 bit dello *State* e i 128 bit della chiave di round corrente.
- **Flessibilità:** Può essere vista come un'operazione colonna per colonna (ogni colonna dello stato contro una parola della chiave) o come una semplice operazione a livello di byte.
- **Inversione:** Per decifrare, si esegue nuovamente lo XOR con la stessa chiave di round, poiché l'operazione XOR è l'inversa di se stessa ($A \oplus B \oplus B = A$).



- **Posizionamento:** Il cifrario inizia e finisce sempre con questa fase. Qualsiasi altra fase all'inizio o alla fine sarebbe inutile, poiché invertibile senza conoscere la chiave.

Sequenza del Cifrario

L'algoritmo segue questa struttura specifica:

1. **Inizio:** Fase preliminare di *Add Round Key*.
2. **Corpo:** 9 round completi (tutte e 4 le fasi).
3. **Conclusione:** Un 10° round che **salta il Mix Columns** e termina con l'ultimo *Add Round Key*.

Struttura della cifratura a flusso

*A differenza dei cifrari a blocco, un cifrario a flusso elabora gli elementi dell'input (bit, byte o parole) in modo **continuo**.*

Un generatore di bit **pseudocasuali** prende una chiave in input e produce uno stream di numeri simili a quelli casuali (**keystream**). Lo stream generato deve essere imprevedibile senza la conoscenza della chiave di input. Per effettuare le operazioni di cifratura e decrittazione, si esegue un'operazione **XOR** tra lo stream di byte del testo in chiaro (M) e lo stream di chiavi (k) per ottenere il testo cifrato (C). Il processo è identico per la decrittazione.

Algoritmo RC4

L'RC4 è un cifrario a flusso a dimensione di chiave variabile con operazioni orientate ai byte. Si basa sull'uso di una permutazione casuale ed è noto per la sua semplicità e velocità (richiede da 8 a 16 operazioni macchina per ogni byte di output). Presenta due fasi:

Fase di Inizializzazione

L'algoritmo inizia preparando uno stato interno tramite una chiave di lunghezza variabile (da 1 a 256 byte):

- **Vettore di Stato S:** Viene inizializzato un vettore S di 256 byte con elementi da 0 a 255 ($S[0] = 0, S[1] = 1, \dots, S[255] = 255$).
- **Vettore Temporaneo T:** Se la chiave K è lunga 256 byte, viene copiata in T. Se è più corta, viene ripetuta ciclicamente fino a riempire i 256 byte di T.
- **Permutazione Iniziale:** Si scorre il vettore S scambiando ogni elemento $S[i]$ con un altro elemento $S[j]$ secondo uno schema dettato da T:

$$j = (j + S[i] + T[i]) \pmod{256}$$

$$\text{Swap}(S[i], S[j])$$

- Dopo questa fase, il vettore S contiene ancora tutti i numeri da 0 a 255, ma permutati casualmente; la chiave originale non viene più utilizzata.

Generazione dello Stream

Una volta inizializzato S, l'algoritmo genera un byte di chiave k alla volta:

1. Si incrementa i e si calcola j basandosi sui valori correnti di S:

$$i = (i + 1) \pmod{256}$$

$$j = (j + S[i]) \pmod{256}$$

2. Si scambiano nuovamente $S[i]$ e $S[j]$.
3. Si calcola l'indice t per estrarre il byte dello stream k:

$$t = (S[i] + S[j]) \pmod{256}$$

$$k = S[t]$$

Cifratura: Il valore k viene XORato con il byte di testo in chiaro.

Decrittazione: Il valore k viene XORato con il byte di testo cifrato.

Modalità di Funzionamento (Block Cipher Modes)

Le modalità di funzionamento definiscono come applicare algoritmi come l'AES a messaggi più lunghi di un singolo blocco.

Modalità Electronic Codebook (ECB)

Il modo più semplice per procedere è quello noto come modalità **electronic codebook (ECB)**, in cui il testo in chiaro viene gestito b bit alla volta e ogni blocco di testo in chiaro viene cifrato utilizzando la stessa chiave. Il termine **codebook** (libro dei codici) viene utilizzato perché, per una data chiave, esiste un testo cifrato unico per ogni blocco di testo in chiaro da b bit. Pertanto, si può immaginare un gigantesco libro dei codici in cui è presente una voce per ogni possibile pattern di testo in chiaro da b bit che mostra il corrispondente testo cifrato.

Con l'ECB, se lo stesso blocco da b bit appare più di una volta nel messaggio, produce sempre lo stesso testo cifrato. Per questo motivo, per messaggi lunghi, la modalità ECB potrebbe non essere sicura. Se il messaggio è altamente strutturato, potrebbe essere possibile per un **crittoanalista** sfruttare queste

regolarità. Ad esempio, se è noto che il messaggio inizia sempre con determinati campi predefiniti, il **crittoanalista** potrebbe avere un numero di coppie testo in chiaro-testo cifrato note con cui lavorare. Se il messaggio presenta elementi ripetitivi, con un periodo di ripetizione multiplo di b bit, questi elementi possono essere identificati dall'analista. Ciò può aiutare nell'analisi o fornire l'opportunità di sostituire o riorganizzare i blocchi.

Per superare le carenze di sicurezza dell'ECB, vorremmo una tecnica in cui lo stesso blocco di testo in chiaro, se ripetuto, produca blocchi di testo cifrato diversi.

Modalità Cipher Block Chaining (CBC)

Nella modalità **cipher block chaining (CBC)**, l'input dell'algoritmo di cifratura è lo XOR del blocco di testo in chiaro corrente e del testo cifrato precedente.

Viene utilizzata la stessa chiave per ogni blocco. In effetti, abbiamo "incatenato" insieme l'elaborazione della sequenza di blocchi di testo in chiaro. L'input della funzione di cifratura per ogni blocco di testo in chiaro non ha una relazione fissa con il blocco di testo in chiaro stesso. Pertanto, i pattern ripetuti di b bit non vengono esposti.

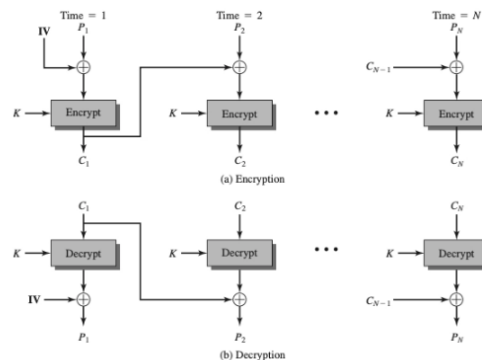


Figure 20.7 Cipher Block Chaining (CBC) Mode

Per la decrittazione, ogni blocco cifrato viene fatto passare attraverso l'algoritmo di decrittazione. Il risultato viene sottoposto a XOR con il blocco di testo cifrato precedente per produrre il blocco di testo in chiaro. Per vedere come funziona, possiamo scrivere:

$$C_j = E(K, [C_{j-1} \oplus P_j])$$

dove $E(K, X)$ è la cifratura del testo in chiaro X utilizzando la chiave K , e $\oplus$ è l'operazione di OR esclusivo (XOR). Allora:

$$\begin{aligned} D(K, C_j) &= D(K, E(K, [C_{j-1} \oplus P_j])) \\ D(K, C_j) &= C_{j-1} \oplus P_j \\ C_{j-1} \oplus D(K, C_j) &= C_{j-1} \oplus C_{j-1} \oplus P_j = P_j \end{aligned}$$

il che verifica il processo di decrittazione.

Per produrre il primo blocco di testo cifrato, un **vettore di inizializzazione (IV)** viene sottoposto a XOR con il primo blocco di testo in chiaro. In decrittazione, l'IV viene sottoposto a XOR con l'output dell'algoritmo di decrittazione per recuperare il primo blocco di testo in chiaro. L'IV deve essere noto sia al mittente che al ricevente. Per la massima sicurezza, l'IV dovrebbe essere protetto così come la chiave. Questo potrebbe essere fatto inviando l'IV tramite cifratura ECB.

Una ragione per proteggere l'IV è la seguente: se un avversario è in grado di ingannare il ricevente spingendolo a usare un valore diverso per l'IV, allora l'avversario è in grado di invertire bit selezionati nel primo blocco di testo in chiaro. Per vederlo, considera quanto segue:

$$\begin{aligned} C_1 &= E(K, [IV \oplus P_1]) \\ P_1 &= IV \oplus D(K, C_1) \end{aligned}$$

Sia $X[i]$ il i -esimo bit della quantità da b bit X . Allora:

$$P_1[i] = IV[i] \oplus D(K, C_1)[i]$$

Quindi, usando le proprietà dello XOR, possiamo affermare:

$$P_1[i]' = IV[i]' \oplus D(K, C_1)[i]$$

dove la notazione "primo" (') indica la **complementazione** del bit (inversione). Ciò significa che se un avversario può cambiare i bit nell'IV in modo prevedibile, i bit corrispondenti del valore ricevuto di P_1 possono essere modificati.

Modalità Cipher Feedback (CFB)

È possibile convertire qualsiasi cifrario a blocchi in un **cifrario a flusso** utilizzando la modalità **cipher feedback (CFB)**. Un cifrario a flusso elimina la necessità di completare (**padding**) un messaggio per renderlo un numero intero di blocchi. Può anche operare in tempo reale. Pertanto, se viene trasmesso un flusso di caratteri, ogni carattere può essere cifrato e trasmesso immediatamente utilizzando un cifrario a flusso orientato ai caratteri.

Una proprietà desiderabile di un cifrario a flusso è che il testo cifrato sia della stessa lunghezza del testo in chiaro. Quindi, se vengono trasmessi caratteri a 8 bit, ogni carattere dovrebbe essere cifrato utilizzando 8 bit. Se si utilizzano più di 8 bit, la capacità di trasmissione viene sprecata. Nello schema CFB, si assume che l'unità di trasmissione sia di s bit; un valore comune è $s=8$. Come nel CBC, le unità di testo in chiaro sono concatenate tra loro, in modo che il testo cifrato di qualsiasi unità di testo in chiaro sia una funzione di tutto il testo in chiaro precedente.

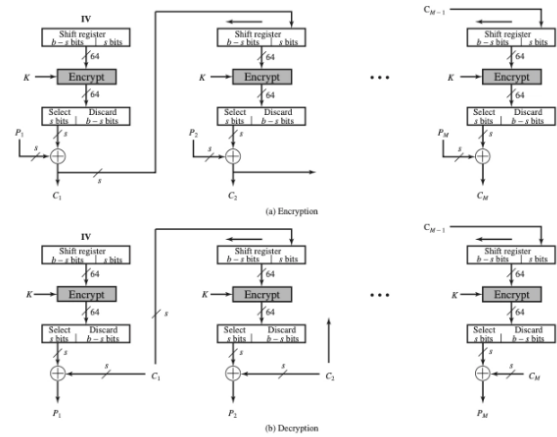


Figure 20.8 s -bit Cipher Feedback (CFB) Mode

Consideriamo prima la cifratura. L'input della funzione di cifratura è uno **shift register** da b bit impostato inizialmente su un vettore di inizializzazione (IV). Gli s bit più significativi (a sinistra) dell'output della funzione di cifratura vengono sottoposti a XOR con la prima unità di testo in chiaro P_1 per produrre la prima unità di testo cifrato C_1 , che viene quindi trasmessa. Inoltre, il contenuto dello **shift register** viene spostato a sinistra di s bit e C_1 viene inserito negli s bit più a destra (meno significativi) dello **shift register**. Questo processo continua finché tutte le unità di testo in chiaro non sono state cifrate.

Per la decrittazione viene utilizzato lo stesso schema, tranne per il fatto che l'unità di testo cifrato ricevuta viene sottoposta a XOR con l'output della funzione di cifratura per produrre l'unità di testo in chiaro. Si noti che viene utilizzata la funzione di **cifratura** (encryption) e non quella di decrittazione.

Modalità Counter (CTR)

Sebbene l'interesse per la modalità **counter** (CTR) sia aumentato recentemente con applicazioni alla sicurezza delle reti ATM e IPsec, questa modalità è stata proposta molto presto.

Viene utilizzato un contatore di dimensione pari a quella del blocco di testo in chiaro. L'unico requisito stabilito dallo standard SP 800-38A è che il valore del contatore deve essere diverso per ogni blocco di testo in chiaro cifrato. Tipicamente, il contatore viene inizializzato a un certo valore e poi incrementato di 1 per ogni blocco successivo (modulo 2^b , dove b è la dimensione del blocco). Per la cifratura, il contatore viene cifrato e poi sottoposto a XOR con il blocco di testo in chiaro per produrre il blocco di testo cifrato; non c'è concatenamento. Per la decrittazione, viene utilizzata la stessa sequenza di valori del contatore, con ogni contatore cifrato sottoposto a XOR con un blocco di testo cifrato per recuperare il blocco di testo in chiaro corrispondente.